

El algoritmo de Chaitin

Asignación de registros por coloreo de grafos

Equipo:	Daniela Nava Martínez · Max Uriel Sánchez Díaz
Voz en off:	Max Uriel Sánchez Díaz
Materia:	Traductores / Compiladores · Prof. Antonio López Jaimes
Duración:	Objetivo 5–7 min (este guión dura aprox. 6.5 min)

Lee cada bloque mientras muestras la diapositiva correspondiente en la web. Los tiempos son una guía; ajusta el ritmo para quedar entre 5 y 7 minutos.

1. Portada

0:00 – 0:25

Hola, somos Daniela Nava Martínez y Max Uriel Sánchez Díaz. En este video vamos a explicar el algoritmo de Chaitin para la asignación de registros mediante coloreo de grafos: qué problema resuelve, cómo funciona, dos ejemplos y nuestras conclusiones.

2. El problema

0:25 – 1:05

El compilador genera código con una cantidad ilimitada de registros virtuales o temporales. Pero el procesador solo tiene un número pequeño de registros físicos, por ejemplo 8, 16 o 32. La asignación de registros decide qué temporales van a registros y cuáles a memoria. La regla clave: dos variables que están vivas al mismo tiempo NO pueden compartir el mismo registro.

3. La idea clave

1:05 – 1:40

La gran idea —desarrollada por Gregory Chaitin y su equipo en IBM a principios de los 80, a partir de una sugerencia de John Cocke— fue ver la asignación de registros como un problema de coloreo de grafos. Cada variable es un nodo. Si dos variables interfieren, o sea viven al mismo tiempo, dibujamos una arista entre ellas. Los registros son colores. Colorear el grafo con k colores, sin que dos nodos vecinos tengan el mismo color, equivale a asignar k registros sin conflictos.

4. Motivación

1:40 – 2:20

¿Qué mejora sobre los métodos anteriores? En clase vimos la asignación local por descriptores de registros: trabaja bloque por bloque, con un método ad-hoc. Es rápida, pero desperdicia registros y mete muchos accesos a memoria. La verdadera aportación de Chaitin no fue simplemente ser global —ya existían esquemas globales, como los basados en prioridad—, sino formular la asignación de registros como un problema de coloreo del grafo de interferencia: un modelo matemático claro y sistemático que además, con el coalescing, elimina copias redundantes. Ojo: colorear es NP-completo, así que Chaitin usa una heurística muy elegante en lugar de buscar la solución óptima.

5. Grafo de interferencia

2:20 – 2:55

Primero necesitamos el grafo de interferencia. Con un análisis de vida (liveness) calculamos, para cada variable, desde dónde hasta dónde está viva. Fíjense en las barras de la izquierda: cuando dos barras se solapan verticalmente, esas variables viven a la vez y por lo tanto interfieren. Cada interferencia se convierte en una arista del grafo de la derecha. Ese grafo es la entrada del algoritmo. Un detalle importante: si dos variables solo coinciden en una instrucción de copia, como b igual a a , no se pone arista entre ellas; esa excepción es la que después permite fusionarlas con el coalescing.

6. Heurística de Kempe

2:55 – 3:30

El corazón del algoritmo es una idea de 1879, la heurística de Kempe. Dice lo siguiente: si un nodo tiene menos de k vecinos, siempre lo podremos colorear, porque sus vecinos usan a lo más $k-1$ colores y siempre sobra al menos uno. Entonces, en la fase Simplify, quitamos repetidamente del grafo cualquier nodo con grado menor que k y lo ponemos en una pila. Al quitar un nodo bajan los grados de sus vecinos, lo que suele permitir quitar más. Después, en la fase Select, vaciamos la pila en orden inverso y a cada nodo le damos un color que no usen sus vecinos: por Kempe, siempre hay uno libre.

7. Fases del algoritmo

3:30 – 4:00

Poniéndolo todo junto, el algoritmo tiene estas fases: Build construye el grafo; Coalesce fusiona copias que no interfieren para eliminar movimientos innecesarios; se calculan los costos de derrame; Simplify apila los nodos fáciles; y Select asigna los registros. Si al final hubo algún derrame real, se inserta el código de guardar y cargar en memoria, y como eso cambia los rangos de vida, se reconstruye el grafo y se repite todo. Cuando ya no hay derrames, terminamos.

8. Derrame (spilling)

4:00 – 4:35

¿Y si en algún momento todos los nodos tienen grado mayor o igual que k ? Nos atoramos: hay que derramar una variable a memoria. ¿Cuál elegir? Chaitin usa la heurística costo entre grado. El costo estima cuántas cargas y almacenamientos costaría derramar esa variable, ponderado por la profundidad de los bucles, porque derramar algo dentro de un bucle es carísimo. El grado es cuántas otras variables estorba. Derramamos el nodo con el menor cociente costo entre grado: barato de derramar y que libera muchas interferencias. Una nota: el Chaitin original derrama de inmediato; Briggs, en 1994, propuso el coloreo optimista, que primero apila el nodo y solo lo derrama si de verdad no encuentra color.

9. Ejemplo 1 — sin derrame

4:35 – 5:25

Primer ejemplo, con k igual a 3 registros. Miren el grafo: los nodos d y e tienen grado menor que 3, así que Simplify los apila primero. Al quitarlos, a , b y c quedan con grado 2, también menor que 3, y se apilan. Con el grafo vacío empieza Select: sacamos de la pila y a cada nodo le damos el registro más bajo que no usen sus vecinos. Todos reciben color: $R1$, $R2$ o $R3$. Resultado: coloreamos con 3 registros y sin ningún derrame. (En la web puedes darle a Reproducir para verlo animado paso a paso.)

10. Ejemplo 2 — con derrame

5:25 – 6:20

Segundo ejemplo, también con 3 registros, pero ahora el grafo es un $K4$: cuatro variables donde todas interfieren entre sí. Un $K4$ necesita 4 colores, pero solo tenemos 3. Al inicio todos los nodos tienen grado 3, así que Simplify se atora y hay que derramar. Calculamos costo entre grado: la variable c tiene el menor cociente, 3 entre 3 igual a 1, así que se marca como derrame. Al quitar c , quedan a , b y d formando un triángulo que sí es 3-coloreable. En Select coloreamos a , b y d , pero cuando toca c sus tres vecinos ya usan $R1$, $R2$ y $R3$, así que c no encuentra color y se confirma el derrame: c vive en memoria. Después se inserta el código de derrame y se reconstruye el grafo, que ahora sí es coloreable.

11. Comparación

6:20 – 6:55

¿Cómo se compara con otros algoritmos? La asignación local por descriptores es muy rápida pero de baja calidad y solo local. Chaitin es global y de alta calidad, pero lento porque el grafo puede ser cuadrático. Briggs mejora a Chaitin con el coloreo optimista, produciendo menos derrames. Y el Linear Scan, de 1999, es casi lineal y muy rápido, aunque de calidad media: por eso se usa en compiladores JIT, donde el tiempo de compilación importa mucho.

12. Conclusiones

6:55 – 7:30

Conclusiones. ¿Cuándo conviene Chaitin? Cuando hay más variables vivas que registros y buscamos código de alta calidad, típico en un compilador optimizador. ¿Es más complejo pero vale la pena? Sí: reduce accesos a memoria y elimina copias, aunque cuesta implementar el análisis de vida y construir el grafo. ¿Hay mejores? Depende del objetivo: Briggs mejora la calidad, George y Appel mejoran el coalescing, y el Linear Scan gana cuando importa la velocidad de compilación. Lo más elegante es que convierte un problema NP-completo en una heurística simple y efectiva.

13. ¿En nuestro traductor?

7:30 – 8:00

¿Lo usaríamos en nuestro traductor? Nuestro traductor hace asignación local por descriptores, como vimos en clase, y para el alcance del curso eso es suficiente. Usaríamos Chaitin si quisiéramos generar código de máquina real con un número fijo de registros y optimizar de verdad el uso del CPU. El costo sería agregar análisis de vida global y la construcción del grafo. En resumen: no es necesario para nuestro proyecto actual, pero sería el siguiente paso natural si convertimos el traductor en un back-end optimizador.

14. Cierre y referencias

8:00 – 8:20

En resumen: Chaitin modela la asignación de registros como coloreo de grafos, usa la heurística de Kempe para simplificar, y derrama a memoria lo que no cabe. Es la base de la asignación de registros moderna. Gracias por su atención.